Inheritance

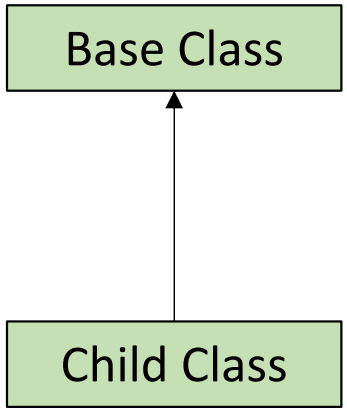
- Inheritance is a fundamental Object Oriented concept.
- Inheritance is declared using the "extends" keyword.
- A class can be defined as a "subclass" of another class.
  - The subclass inherits all data attributes of its superclass
  - The subclass inherits all methods of its superclass
  - The subclass inherits all associations of its superclass
- The subclass can:
  - Add new functionality
  - Use inherited functionality
  - Override inherited functionality

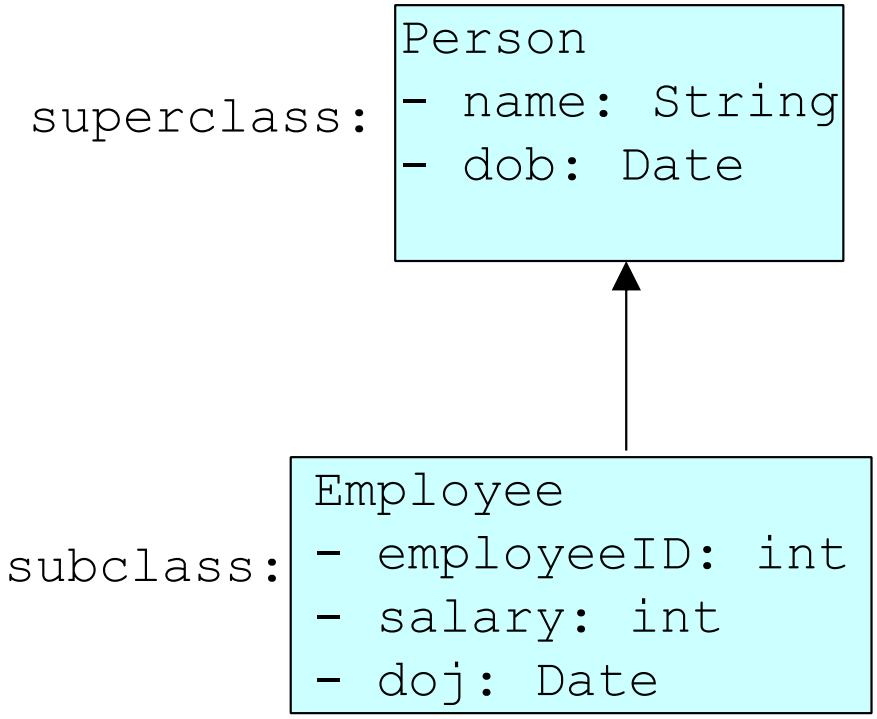
Inheritance terms

- **Superclass**, **base class**, **parent class**: terms to describe the parent in the relationship, which shares its functionality
- **Subclass**, **derived class**, **child class**: terms to describe the child in the relationship, which accepts functionality from its parent
- **Extend**, **inherit**, **derive**: become a subclass of another class

Types of Inheritance

1. Single Inheritance: One class extends only one class.

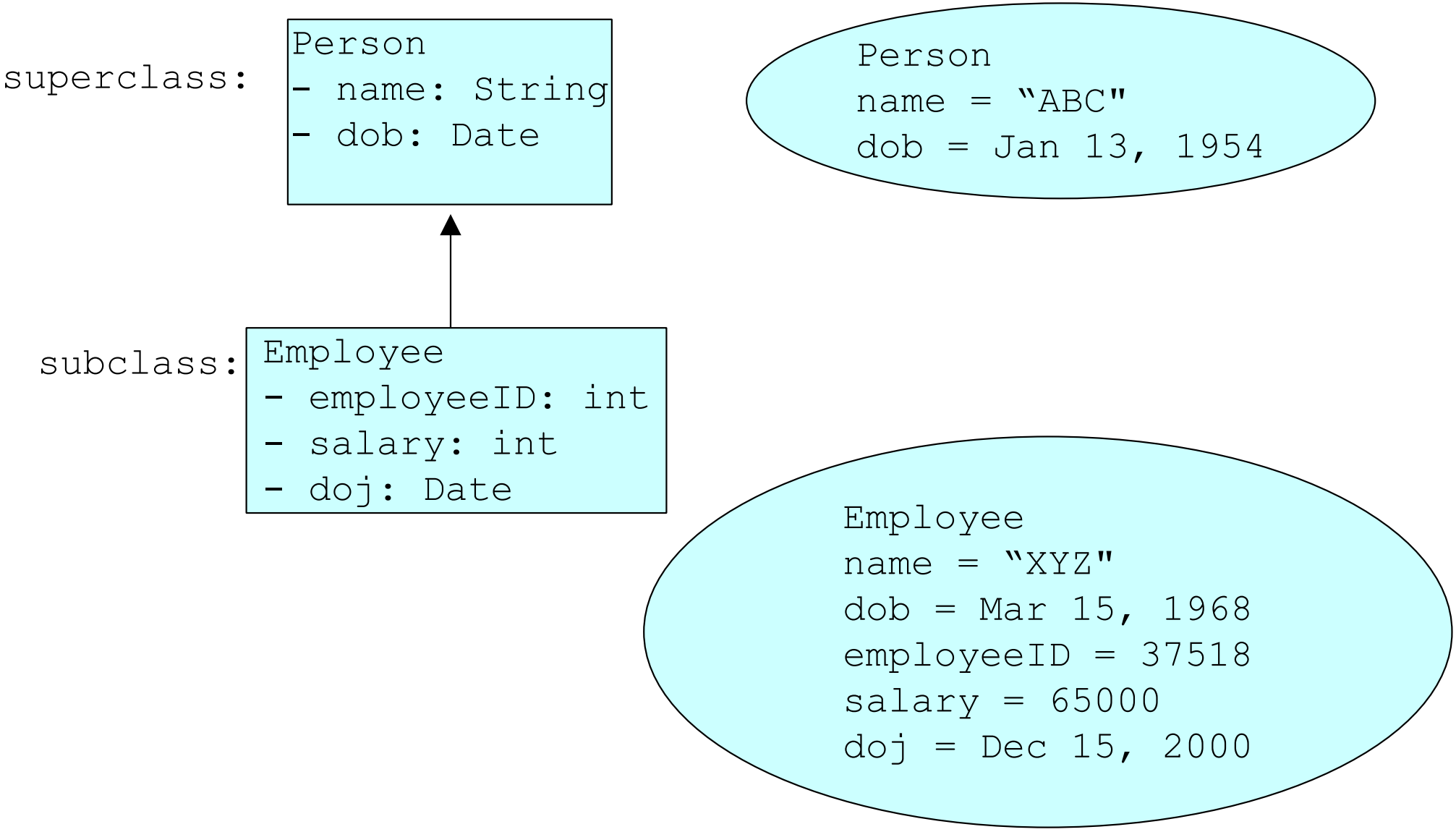




Types of Inheritance

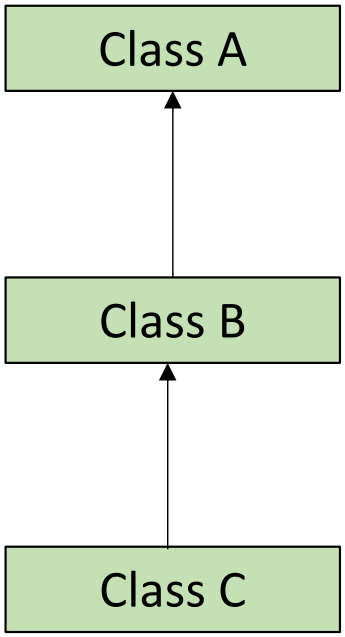
```
class Person
{   String name;
    Date dob;
    [...]
}
class Employee extends Person
{    int employeID;
     int salary;
     Date doj;
     [...]
}
…
…
…
```

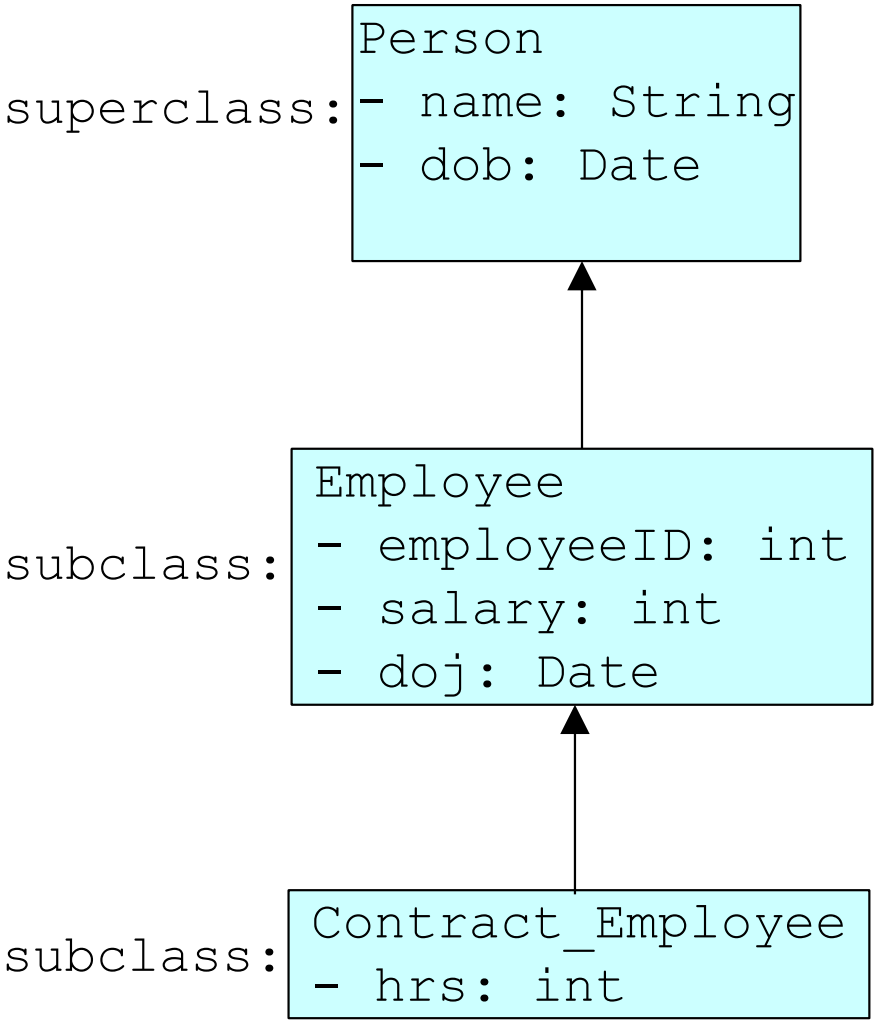
Types of Inheritance



Types of Inheritance

2. Multilevel Inheritance: Hierarchy of single level inheritance.

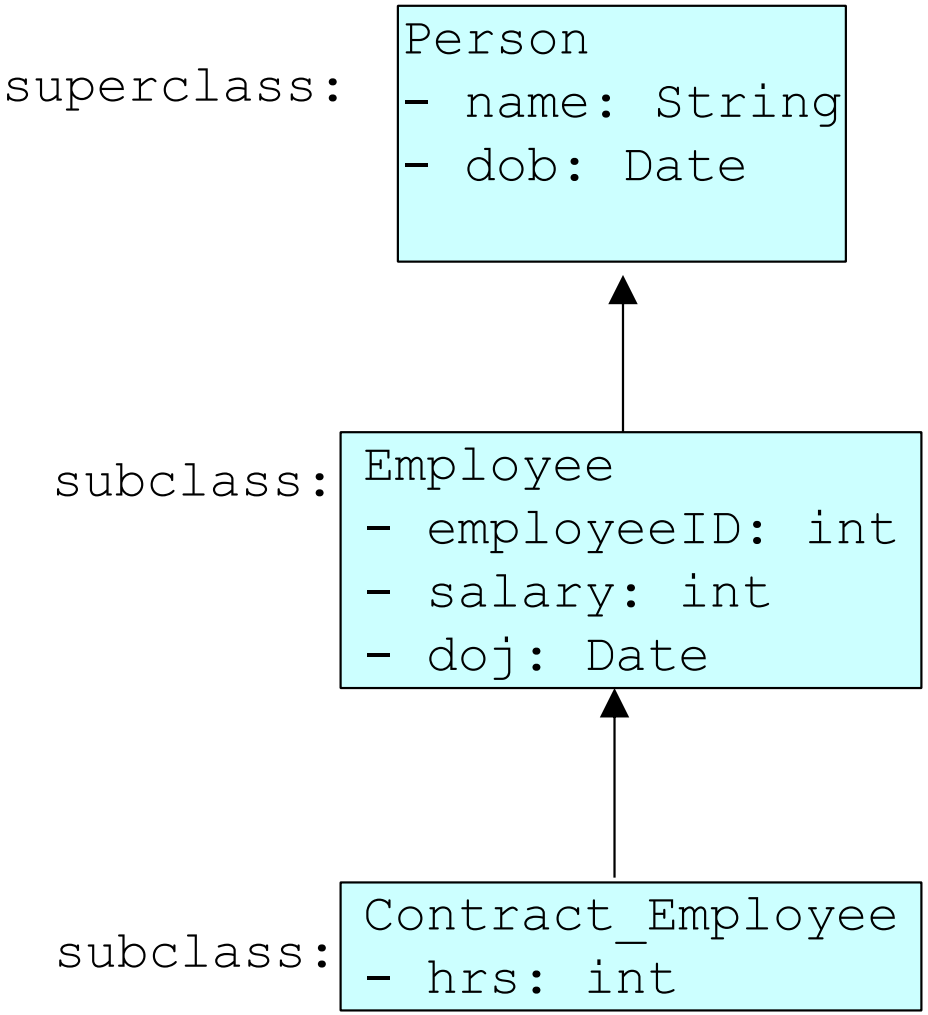




Types of Inheritance

```
class Person
{   String name;
    Date dob;
    [...]
}
class Employee extends Person
{    int employeID;
     int salary;
     Date doj;
     [...]
}
class Contract_Employee extends Employee
{   int hrs;
    [...]
 }
…
```

Types of Inheritance



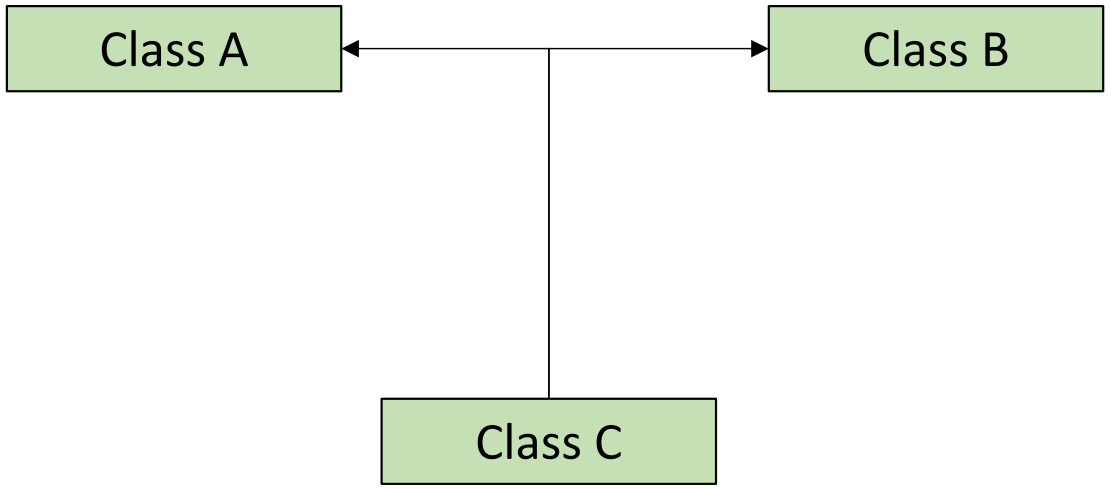
Person
name = "ABC"
dob = Jan 13, 1954

Employee
name = "XYZ"
dob = Mar 15, 1968
employeeID = 37518
salary = 65000
doj = Dec 15, 2000

Contract_Employee
name = "DEF"
dob = Mar 21, 1975
employeeID = 37529
salary = 75000
doj = Dec 15, 2002
hrs = 15

Types of Inheritance

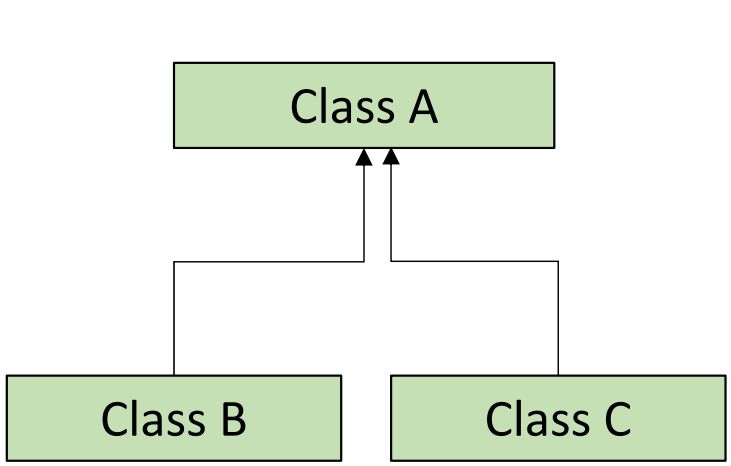
3. Multiple Inheritance: One class extending from more than one class.

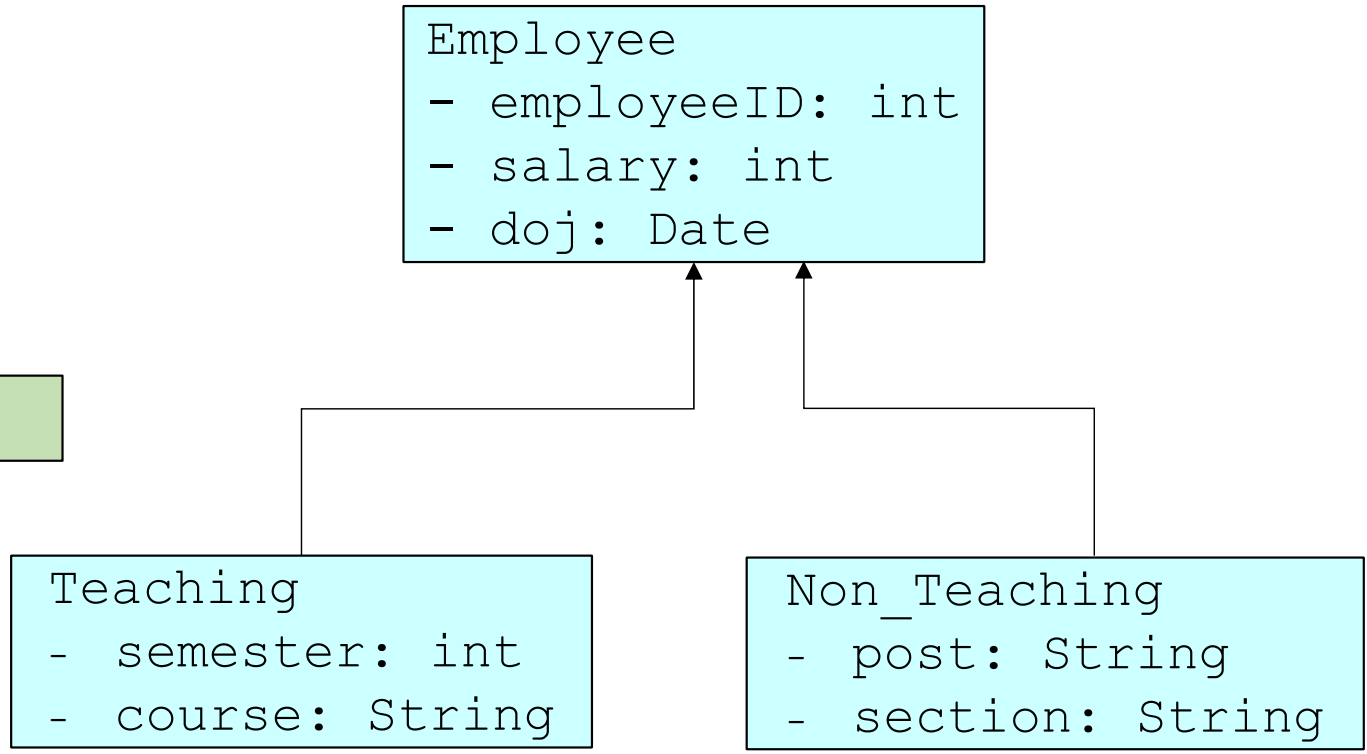


Java does not support multiple inheritance amongst classes.
It can be achieved by using interfaces.

Types of Inheritance

4. Hierarchical Inheritance: More than one class can inherit from a single class.

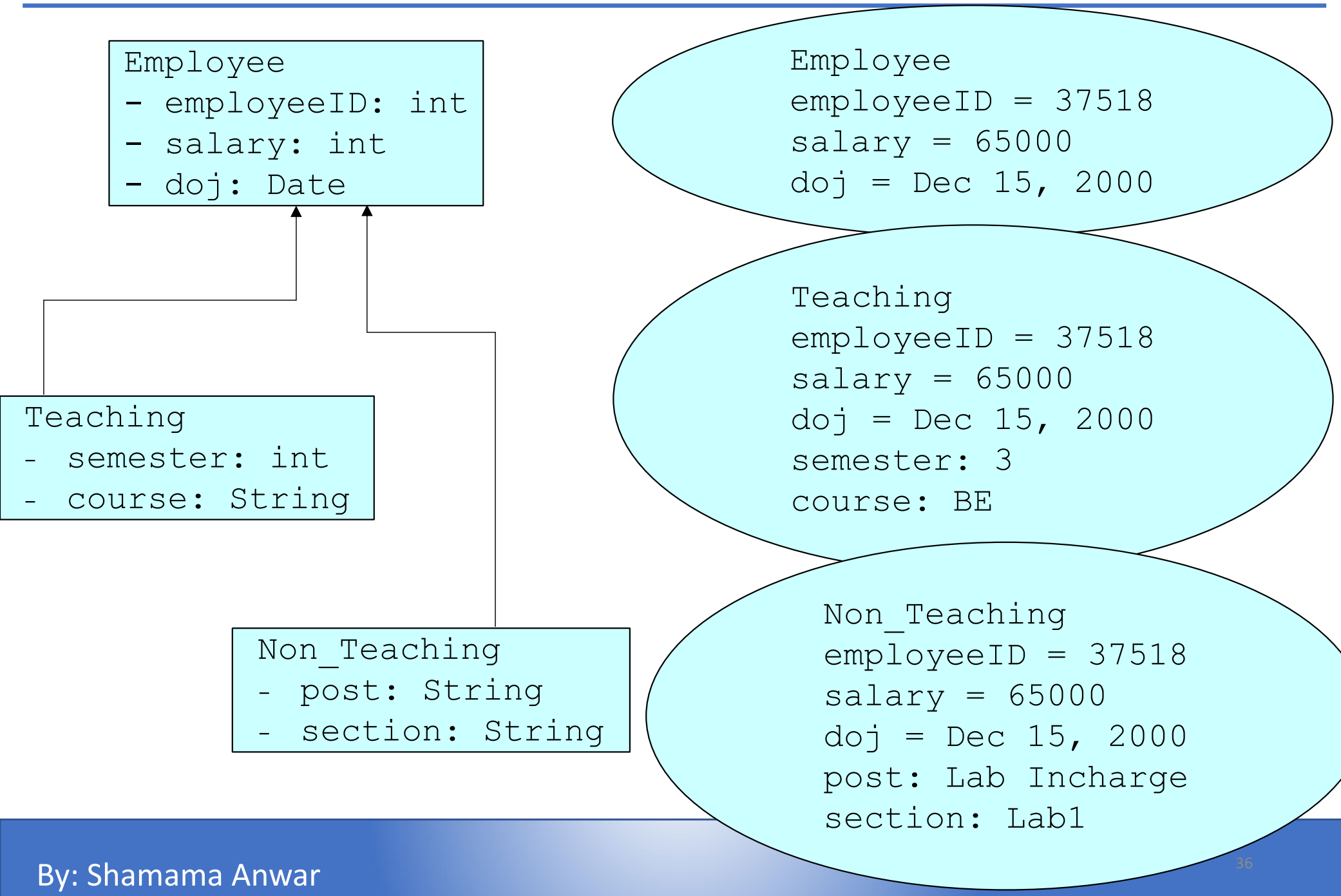




Types of Inheritance

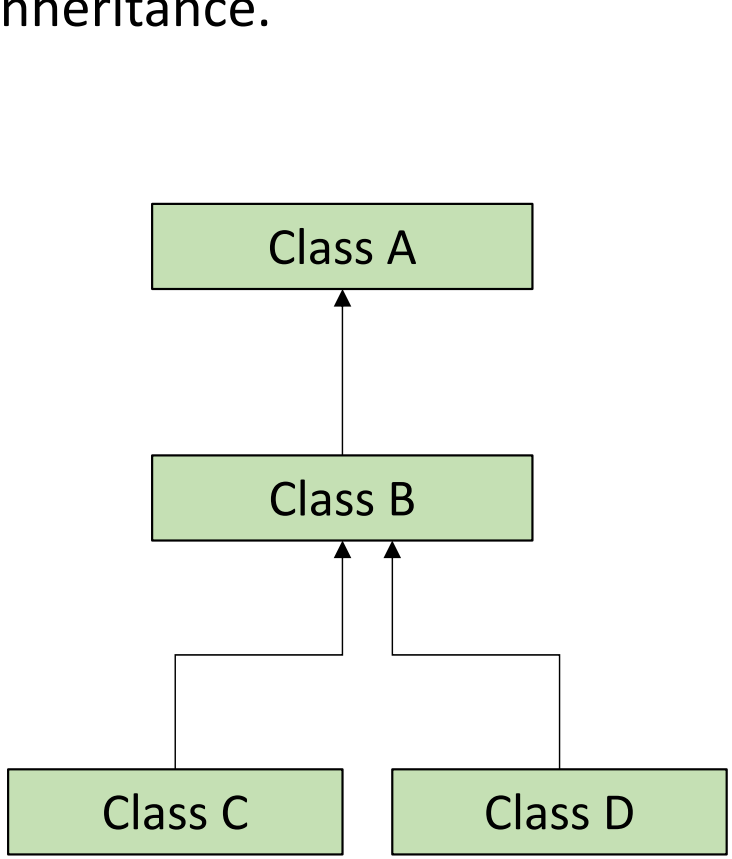
```
class Employee
{  int employeeid;
   int salary;
   Date doj;
   [...]
}
class Teaching extends Employee
{   int semester;
    String course;
    [...]
}
class Non_Teaching extends Employee
{  String post;
   String section;
   [...]
 }
…
```

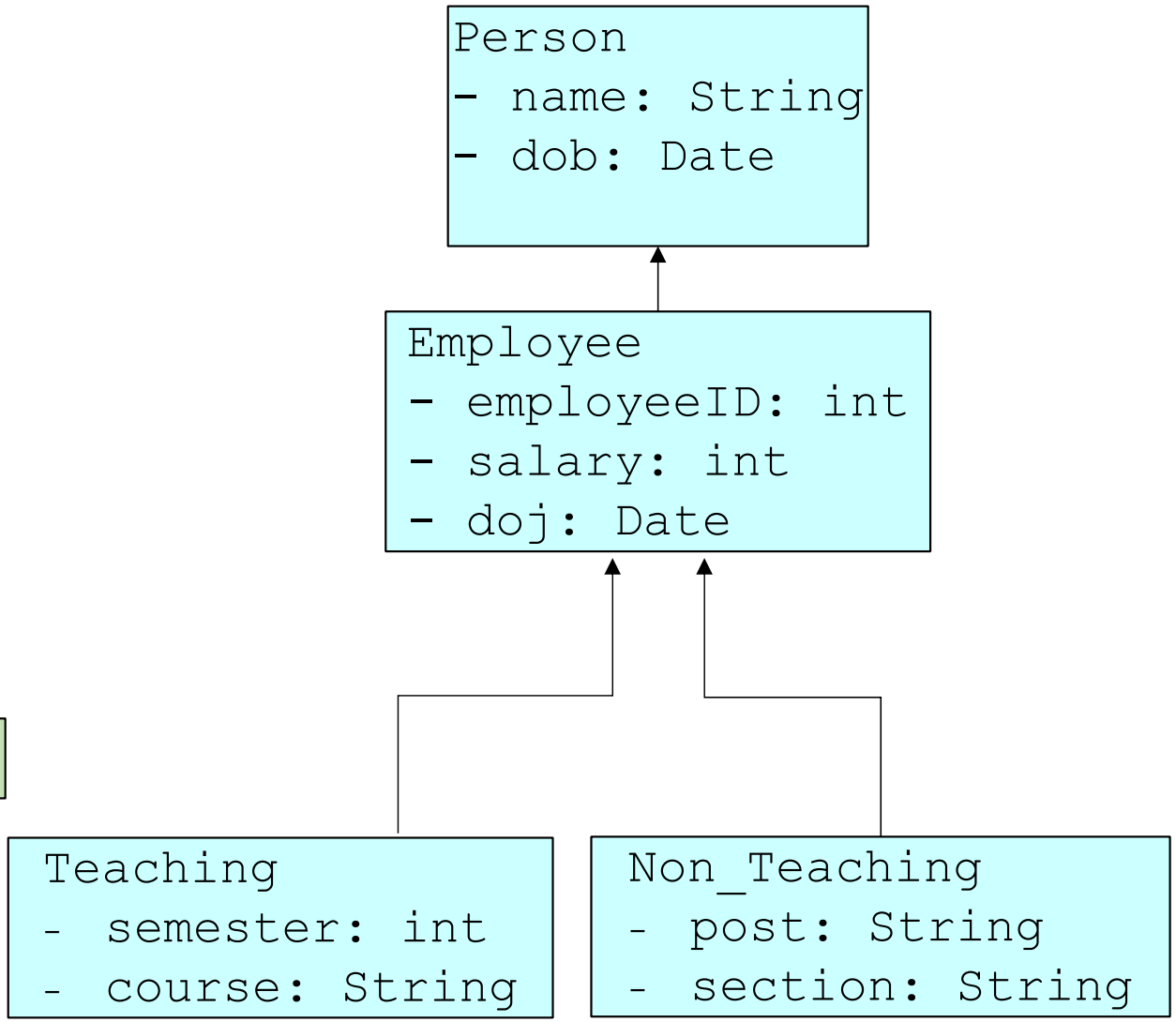
Types of Inheritance



Types of Inheritance

5. Hybrid Inheritance: Combination of single level, multilevel or hierarchical inheritance.



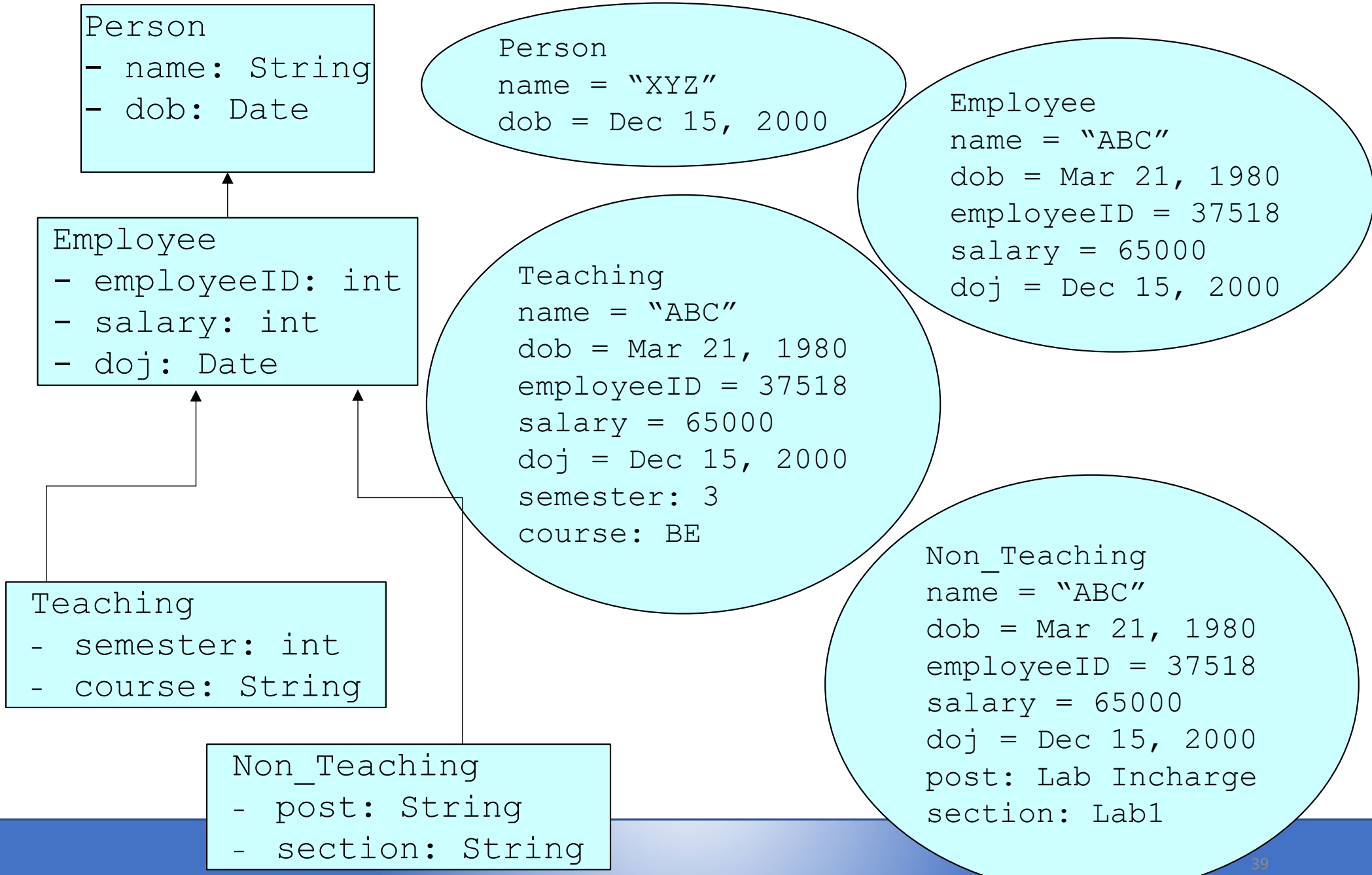


Types of Inheritance

```
class Person
{   String name;
    Date dob;
    [...]
}
class Employee extends
Person
{   int employeeid;
    int salary;
    Date doj;
    [...]
}
```

```
class Teaching extends Employee
{    int semester;
     String course;
     [...]
}
class Non_Teaching extends
Employee
{   String post;
    String section;
    [...]
 }
…
```

Types of Inheritance



Inheritance: Example 1

```
class Person
{  String name;
   Date dob;
   void getdata() { Scanner sc = new Scanner(System.in);
    System.out.println("Enter name:");
    name = sc.next();
    System.out.println("Enter DOB (dd mm yyyy):");
    int dd = sc.nextInt();
    int mm = sc.nextInt();
    int yy = sc.nextInt();
    dob = new Date(yy, mm, dd); }
   void display ()
   { System.out.println("Name:"+name);
     System.out.println("DOB:"+dob);
    }
 }
```

Inheritance: Example 1

```
class Employee extends Person
{    int employeID;
     int salary;
     Date doj;
   void getdata1() { Scanner sc = new Scanner(System.in);
     System.out.println("Enter employee id:");
     employeID = sc.nextInt();
     System.out.println("Enter DOJ (dd mm yyyy):");
     int dd = sc.nextInt();   int mm = sc.nextInt();
     int yy = sc.nextInt();   dob = new Date(yy, mm, dd);
     System.out.println("Enter Salary:");
     salary = sc.nextInt();
 }
    void display1()
    { System.out.println("EmployeeID:"+employeID);
      System.out.println("Salary:"+salary);
      System.out.println("DOJ:"+doj); } }
```

Inheritance: Example 1

```
class Test
{ public static void main(String args[])
   { Employee e1 = new Employee();
     e1.getdata();
     e1.getdata1();
     e1.display();
     e1.display1();
   }
 }
```

Inheritance: Method Overriding

- When a method in a subclass has the same name and type signature as a method in its super class, then the method in the subclass is said to override the method in superclass.

- This feature supports polymorphism.

```
class A
{ int i = 0;
  void f1(int k) { i = k;
   System.out.println("In Superclass:"+i); }
 }
class B extends A
{ void f1(int k) { i = 2*k;
   System.out.println("In Subclass:"+i);  }
}
........
B b = new B(); b.f1(3);
```

Inheritance: `super` keyword

- Super keyword refers to the parent class.

- It has three purposes:
  - For calling the methods of the superclass.
  - For accessing the member variables of the superclass.
  - For invoking the constructors of the superclass

Inheritance: `super` keyword

- Calling the methods of the superclass

```
class A
{  void show()
    {  System.out.println("In Superclass:"); }
 }
class B extends A
{ void show()
   {  super.show();
      System.out.println("In Subclass:");  }
}
class test
{  public static void main(String args[])
    {  B b = new B();
       b.f1();
    }
}
```

Inheritance: Example 2

```
class Person
{  String name;
   Date dob;
   void getdata() { Scanner sc = new Scanner(System.in);
    System.out.println("Enter name:");
    name = sc.next();
    System.out.println("Enter DOB (dd mm yyyy):");
    int dd = sc.nextInt();
    int mm = sc.nextInt();
    int yy = sc.nextInt();
    dob = new Date(yy, mm, dd); }
   void display ()
   { System.out.println("Name:"+name);
     System.out.println("DOB:"+dob);
    }
 }
```

Inheritance: Example 2

```
class Employee extends Person
{    int employeID;
     int salary;
     Date doj;
   void getdata() { super.getdata();
     Scanner sc = new Scanner(System.in);
     System.out.println("Enter employee id:");
     employeID = sc.nextInt();
     System.out.println("Enter DOJ (dd mm yyyy):");
     int dd = sc.nextInt();    int mm = sc.nextInt();
     int yy = sc.nextInt();    dob = new Date(yy, mm, dd);
     System.out.println("Enter Salary:");
     salary = sc.nextInt(); }
    void display () { super.display();
      System.out.println("EmployeeID:"+employeID);
      System.out.println("Salary:"+salary);
      System.out.println("DOJ:"+doj); } }
```

Inheritance: Example 2

```
class Test
{ public static void main(String args[])
   { Employee e1 = new Employee();
     e1.getdata();
     e1.display();
    }
 }
```

Inheritance: `super` keyword

- **For accessing the member variables of the superclass**

```
class A
{ int b = 30;
}
class B extends A
{ int b = 12;
  void show()
   { System.out.println("Subclass variable:"+b);
     System.out.println("Superclass variable:"+super.b);
   }
}
class test
{  public static void main(String args[])
    {  B b = new B();
       b.show();
    }
}
```

Inheritance: `super` keyword

- **For invoking the constructors of the superclass**

```
class A
{ A() { System.out.println("Constructor A"); } }
class B extends A
{ B()  {  System.out.println("Constructor B"); } }
class C extends B
{ C()  {  System.out.println("Constructor C"); } }
class test
{  public static void main(String args[])
    {  C c = new C();
     }
}
```

super() is added in each class constructor automatically by compiler if there is no super() written

Inheritance: `super` keyword

- **For invoking the constructors of the superclass**

```
class A
{ A() { System.out.println("Constructor A"); } }
class B extends A
{ B()  {  System.out.println("Constructor B"); } }
class C extends B
{ C()  {  System.out.println("Constructor C"); } }
class test
{  public static void main(String args[])
    {  C c = new C();
     }
}
```

super() is added in each class default constructor automatically by compiler if there is no super() written

Inheritance: `super` keyword

- **For invoking the constructors of the superclass**

```
class A
{ A() { System.out.println("Constructor A"); } }
class B extends A
{ B()  {  System.out.println("Constructor B"); }
  B(int a){ a++; System.out.println("Constructor B"+a);}}
class C extends B
{ C()  { super(1);
         System.out.println("Constructor C"); } }
class test
{  public static void main(String args[])
    {  C c = new C();
     }
}
```

It is mandatory for a `super` statement in a constructor to be the first statement within the constructor.

Inheritance: Example 3

```
class Person
{   String name;
    String dob;
    Person(){ name = “”; dob = “”; }
    Person(String x, String y)
     { name = x;  dob = y;  }
    void display() {…… }
 }
class Employee extends Person
{   int employeID;
    int salary;
    String doj;
    Employee() { employeID = 0; salary = 0; doj = “”; }
    Employee(String n, String b, int id, int sal, String j)
     { Person(n,b); employeID = id; salary = sal; doj = j;}
    void display() {…….}
}
```

Inheritance: Example 3

```
class Test
{ public static void main(String args[])
   {
     //Get the input values from the user
Employee e1 = new Employee("Abc", "12/03/1980", 1034,
70000, "07/12/2010");
     e1.display();
    }
 }
```

`final` keyword

- To declare a constant (used with variable and argument declaration)
- To disallow method overriding (used with method declaration)
- To disallow inheritance (used with class declaration)

```
class Test
{ final void show()
    { System.out.println("Superclass show"); }
 }
class Test1 extends Test
{  void show() { System.out.println("Subclass show"); }
 }
class demo
 { Test1 obj = new Test1();
   obj.show();
}
```

`final` keyword

- To disallow inheritance (used with class declaration)

```
final class Test
{void show()
   { System.out.println("Superclass show"); }
 }
class Test1 extends Test
{  void show() { System.out.println("Subclass show"); }
 }
class demo
 { Test1 obj = new Test1();
   obj.show();
}
```

abstract keyword

- The `abstract` keyword can be used for defining methods and classes.
- A method that has been declared but not defined is called an abstract method.

```
public abstract void show();
```

- Any class containing an abstract method is an abstract class.

```
abstract class MyClass {...}
```

- An abstract class is incomplete and hence cannot be instantiated.
- An abstract class can be extended but
  - If the subclass defines all the inherited abstract methods, it is "complete" and can be instantiated
  - If the subclass does *not* define all the inherited abstract methods, it too must be abstract

abstract keyword

- A class can be made `abstract` even if it does not contain any abstract methods. This prevents the class from being instantiated.

- Abstract classes are good for defining a general category (template) containing specific functions.

abstract keyword: Example 1

```
abstract class Shape{  abstract void draw(); }
class Rectangle extends Shape{
void draw(){System.out.println("drawing rectangle");}
}
class Circle extends Shape{
void draw(){System.out.println("drawing circle");}
}

class TestAbstraction1{
public static void main(String args[]){
Circle s=new Circle();
s.draw();
}
}
```

abstract keyword: Example 2

```
public abstract class Employee
{ private String name, address;
  private int number;
  public Employee( ) {. . . . }
  public Employee(String n, String add, int num)
    { . . . }
 public double computePay()
 { System.out.println("Inside Employee computePay");
    return 0.0; }
 public void mailCheck()
 { System.out.println("Mailing to " + this.name +
   " " + this.address); }
 public String getName() { return name; }
 public String getAddress() { return address; }
 public void setAddress(String newAddress)
 { address = newAddress; }
 public int getNumber() { return number; } }
```

abstract keyword: Example 2

```
public class AbstractDemo
{ public static void main(String [] args)
 { Employee e = new Employee(“Abc”, “Ranchi”, “2314”);
   System.out.println("\n Call mailCheck using Employee
reference--");
e.mailCheck(); } }
```

abstract keyword: Example 3

```
public abstract class Employee
{ private String name, address;
  private int number;
  public Employee( ) {. . . . }
  public Employee(String n, String add, int num)
    { . . . }
 public double computePay()
 { System.out.println("Inside Employee computePay");
    return 0.0; }
 public void mailCheck()
 { System.out.println("Mailing to " + this.name +
   " " + this.address); }
 public String getName() { return name; }
 public String getAddress() { return address; }
 public void setAddress(String newAddress)
 { address = newAddress; }
 public int getNumber() { return number; } }
```

abstract keyword: Example 3

```
public class Salary extends Employee
{ private double sal;
  Salary(String n, String add, int num, double sal)
  { super(n, add, num);
    setSalary(salary); }
 public void mailCheck()
 {System.out.println("Within mailCheck of Salary class ");
  System.out.println("Mailing to " + getName() + " with
  salary " + sal); }
 public double getSalary() { return sal; }
 public void setSalary(double newSal)
 { if(newSal >= 0.0) { sal = newSal; } }
```

abstract keyword: Example 3

```
public class AbstractDemo
{ public static void main(String [] args)
{ Salary s = new Salary(“Abc", "UP", 1243, 3600.00);
  System.out.println("Call mailCheck using Salary
reference --");
 s.mailCheck();
} }
```

Program

Create an abstract class `Accounts` with the following details:

- Data members: `balance, accountNumber accountName, address`
- Methods: `withdrawl( )` – abstract, `deposit( )` – abstract
  `display( )` – to show the balance of the account

Create a subclass of this class `SavingAccount` and add the following details:

- Data members: `rateOfInterest`
- Methods: `calulateAmount(),`
  `display()` – to display rate of interest with new balance and full account holder details

Create another subclass of the `Accounts` class, `CurrentAccount` with the following:

- Data Members: `overdraftlimit`
- Methods: `display()` – to show overdraft limit along with the full account holder details.

Create objects of both the classes and use appropriate constructors.

Interfaces

- In Java, only single inheritance is permitted. However, Java provides a construct called as an interface which can be implemented by a class.
- Interfaces are similar to abstract classes .
- A class can implement any number of interfaces. In effect using interfaces gives us the benefit of multiple inheritance.
- Interfaces are compiled into bytecode just like classes.
- Interfaces cannot be instantiated.
- Interfaces can contain only `abstract public` methods. (by default)
- Interfaces can contain only `public, final` and `static` variables. (by default)

Interfaces

- An interface is similar to an abstract class with the following exceptions:
  - All methods defined in an interface are abstract. Interfaces can contain no implementation.
  - Interfaces cannot contain instance variables. However, they can contain public static final variables (ie. constant class variables)

- Interfaces are declared using the "interface" keyword

- Interfaces are more abstract than abstract classes

- Interfaces are implemented by classes using the "implements" keyword.

Interface – Example 1

```
interface calculator
 {
   int add(int a, int b);
   int sub(int a, int b);
   int mult(int a, int b);
   int div(int a, int b);
 }

class demo implements calculator
 {
   public int add(int a, int b){ return a+b; }
   public int sub(int a, int b){ return a-b; }
   public int mult(int a, int b){ return a+b; }
   public int div(int a, int b){ return a/b; }
 }
```

Interface – Example 2

```
interface area
 {   double PI = 3.14;
     int sq(int a);
     int rect(int a, int b);
     double cir(double r);
 }
class demo implements area
 {
   public int sq(int a){ return a*a; }
   public int rect(int a, int b){ return a*b; }
   public double cir(double r){ return area.PI*r*r; }
}
```

Extending Interfaces

- Just like normal classes, interfaces can also be extended.

- An interface can inherit another interface using the keyword extends (not implements)

```
interface A   { void showA(); }

interface B extends A  { void showB(); }

class demo implements B
{public void showA()
 {System.out.ptintln("Overriden method of Interface A"); }
 public void showB()
 {System.out.println("Overriden method of Interface B"); }
 public static void main(String args[])
 { demo d = new demo();
   d.showA();
   d.showB();
  } }
```

Interfaces

```
interface A   { . . . . }

interface B  { . . . . . }

interface C extends B {. . . . }

interface D extends A, B { . . . . }
```

```
class A   { . . . . }

interface B extends A { . . . . }
```

```
interface A   { . . . . }

class B extends A { . . . . }
```

Interfaces

```
interface A   { . . . . }

class B implements A  { . . . . . }
```

```
interface A   { . . . . }

interface B { . . . . }

class C implements A, B { . . . . .}
```

```
interface A   { . . . . }

class B { . . . . .}

class C extends B implements A { . . . . .}
```

Interfaces

```
interface A   { . . . . }

interface B { . . . . . }

class C { . . . .  }

class D extends C implements A, B { . . . . .}
```

Interfaces - Example

1. Design an interface Queue with the following methods:
    - Add elements
    - Delete elements
    - Check if queue is empty

2. Design an interface with a method reversal. This method takes a string as its input and returns the reversed string.

Interface vs. Abstract Class

Interface	Abstract Class
A class can inherit any number of interfaces	A class can inherit only one class
`implements` keyword is used	`extends` keyword is used
All methods are `public` and `abstract`	No restriction
All variables are `public`, `static` and `final`	No restrictions
Interfaces has no method implementation at all	Abstract classes may have implementation of methods.
No constructors	Can have constructors

Packages

- Packages are a way to organize files into different directories according to their functionality, usability as well as category.

- A Java package is a Java programming language mechanism for organizing classes.

- Java source files belonging to the same category or providing similar functionality can include a **package** statement at the top of the file to designate the package for the classes the source file defines.

- Packaging also help us to avoid class name collision when we use the same class name as that of others.

```
package My;
class A
{ . . . . }
```

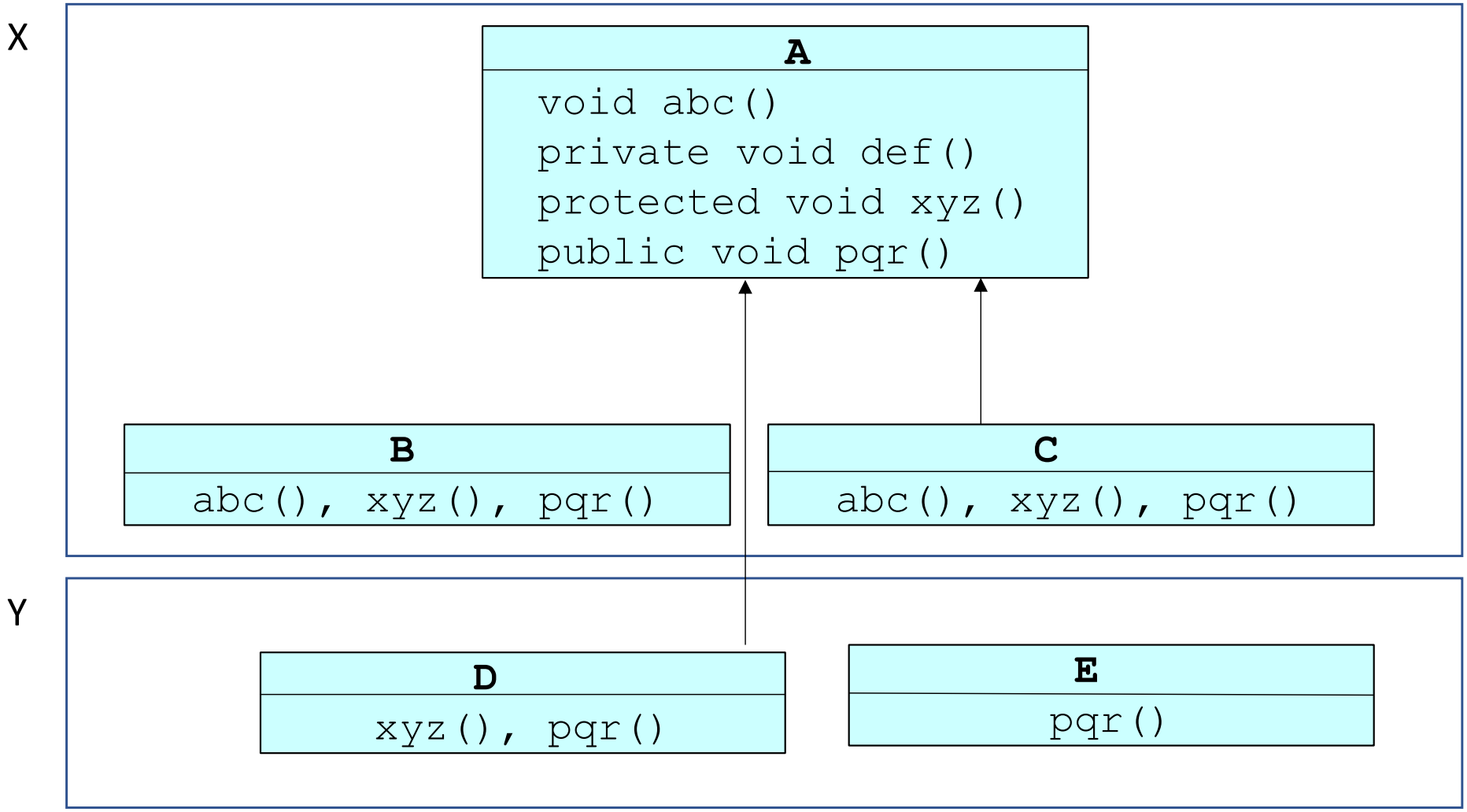
Packages

```
package My;
public class example
 { public int fact(int a)
    { if(a==1)
       return 1;
      else
       return a*fact(a-1); } }
```

```
package My1;
import My.*;
class demo
 { public static void main(String args[])
    {  example e = new example();
       System.out.println("Factorial ="+e.fact(6));
     }
  }
```

Access Modifiers

	Private	Default	Protected	Public
Same class				
Same package sub class				
Same package non subclass				
Different package subclass				
Different package non subclass				


X
A
void abc()
private void def()
protected void xyz()
public void pqr()
B
abc(), xyz(), pqr()
C
abc(), xyz(), pqr()
Y
D
xyz(), pqr()
E
pqr()